Summary in Graph

Exam Summary (GO Classes GATE CS | Data Structures | Weekly Quiz 3)

Qs. Attempted:	0 (0 + 0)	Correct Marks:	0 (0 + 0)
Correct Attempts:	0 (0 + 0)	Penalty Marks:	0 (0 + 0)
Incorrect Attempts:	0 (0 + 0)	Resultant Marks:	0 (0 + 0)

Total Questions:	10 (0 + 10)
Total Marks:	20 (0 + 20)
Exam Duration:	30 Minutes
Time Taken:	0 Minutes

Individual Question Statistics

Select Metric: Marks Obtained

Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8 Q9 Q10

* Average Stats based on first-attempt stats of Top 10% (27) out of 267 users

EXAM RESPONSE EXAM STATS FEEDBACK

Technical

Q #1 Multiple Choice Type Award: 2 Penalty: 0.67 Data Structures

Consider the implementation of a stack of which is based on dynamic array allocation using malloc in C programming. This implementation allows us to push more elements than the current array capacity. It works as follows.

Suppose the array is full, then to push the next element we first allocate a new array of larger size using malloc, copy all elements to the new array, and free up the old array memory.

When a dynamic array-based stack becomes full, you increase the array size by the following sequence $k, 2k, 3k, 4k, \ldots$ for some positive constant k. Which means the first time we increase the size by k, next time by $2k$, and so on.

If you have an empty stack that uses an array whose initial size is k, and you perform n pushes (assume that $n > k$). What is the total cost complexity of executing n push operations?
Let cost of expanding from k to $2k$ has $2k$ cost, similarly cost of expanding from $2k$ to $3k$ has $3k$ cost.

A. $O\left(n^2/k\right)$

B. $O(n/k)$

C. $O\left(n^2k\right)$

D. $O(nk)$

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #2 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Given a circular array-based queue capable of holding 100 objects. Suppose the queue is initially empty, and then objects are put into the queue at the rate of 10 per minute while meantime they are processed and removed from the queue at the rate of 5 per minute. After 120 elements have been added to the queue, which of the following is true?

A. You can't add 120 elements to an array holding 100 entries.

B. There will be 60 elements in the queue, 20 of them at the front of the array where the queue started, and 40 at the other end.

C. There will be 60 elements in the queue, 30 of them at the front of the array where the queue started, and 30 at the other end.

D. There will be 60 elements in the queue, 40 of them at the front of the array where the queue started, and 20 at the other end.

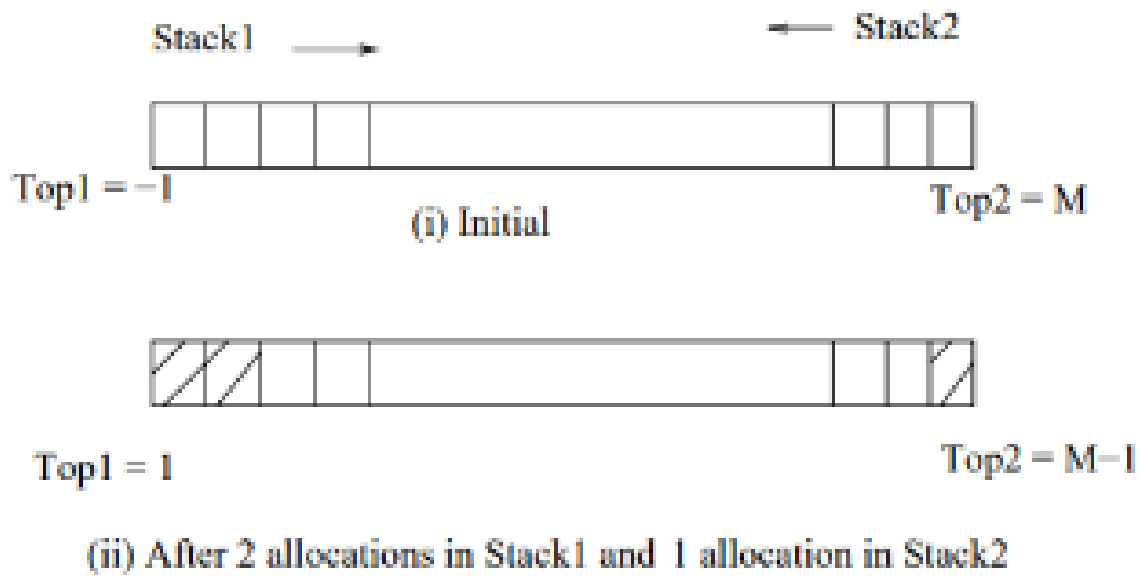
Your Answer: | Correct Answer: B | Not Attempted | Time taken: 00min 02sec | Discuss

Q #3 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Consider the memory allocation of M blocks.

Stack1 growing from block 0 up toward block $\mathrm{M}-1$ and Stack2 growing from block $\mathrm{M}-1$ down to block 0.

Let Top1 (initialized to -1) and Top2 (initialized to M) be the tops of the stacks. For example, after 2 calls to PushStack1 (data) that allocates elements in Stack1 followed by 1 call to PushStack2 (data) that allocates elements in Stack2, the memory that was initially all free blocks now has 3 allocated blocks as per the figure:



If Stack1 was replaced by a circular queue with Front and Rear pointers, then which of the following is true regarding memory utilization?

A. you may not be able to add/push into the stack even when there are some free blocks in the memory.

B. you may not be able to add/enqueue into the queue even when there are some free blocks in the memory.

C. Stack and queue both can be implemented with efficient memory utilization.

D. This implementation is not possible.

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #4 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Suppose that we want to design a data structure called Mx-Queue that supports ONLY three operations as follows:

Enqueue(x) – It is a standard queue operation that equeue element x into the queue.

Dequeue() – It is a standard queue operation that dequeue from front of the queue.

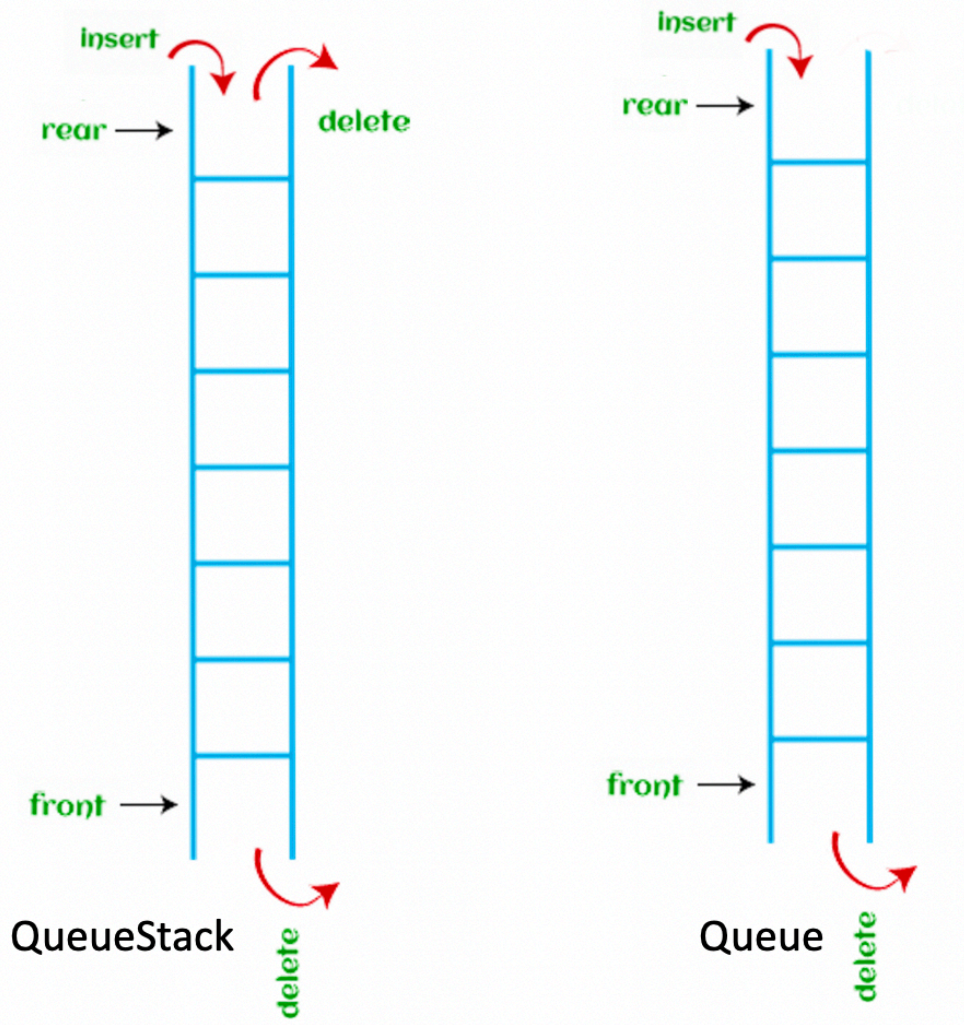
FIND-MAX() – It is one extra operation that Mx-Queue supports which finds the maximum element from MX-Queue without removing it in $O(1)$ time.

To fulfill the purpose we use one more auxiliary data structure along with a standard queue.

There are two choices of auxiliary data structures are available to us. (See figure for visual representation)

- QueueStack
- Queue

QueueStack is a data structure that has the special privilege of removing elements from the rear of the queue.



we call the following functions Insert() and Delete() to add or remove elements into or from Mx-Queue.

```
Insert(Mx-Queue,x):
   Enqueue(Mx-Queue, X)
   //Do something with auxiliary data structure
End


Delete(Mx-Queue):
    x = Dequeue(Mx-Queue)
    //Do something with auxiliary data structure
    Return x
End
```

One of the Insert or Delete works in linear time and the other one works in constant time.

Choose the correct option.

- $S1$: We can implement Mx-Queue by using one Queue and QueueStack data Structure.
- $S2$: We can implement Mx-Queue by using two Queue data Structures.

A. $S1$ is correct but $S2$ is incorrect
B. $S1$ is incorrect but $S2$ is correct
C. Both are correct
D. Both are incorrect

Your Answer: | Correct Answer: A | Not Attempted | Time taken: 00min 00sec | Discuss

Q #5 | Multiple Select Type | Award: 2 | Penalty: 0 | Data Structures

Imagine four railroad cars positioned on the input side of the track in the figure below, numbered $1, 2, 3,$ and 4, from left to right. Suppose we perform the following sequence of operations (which is compatible with the direction of the arrows in the diagram and does not require cars to "jump over" other cars):

a. move car 1 into the stack;
b. move car 2 into the stack;
c. move car 2 into the output;
d. move car 3 into the stack;
e. move car 4 into the stack;
f. move car 4 into the output;

g. move car 3 into the output;
h. move car 1 into the output.

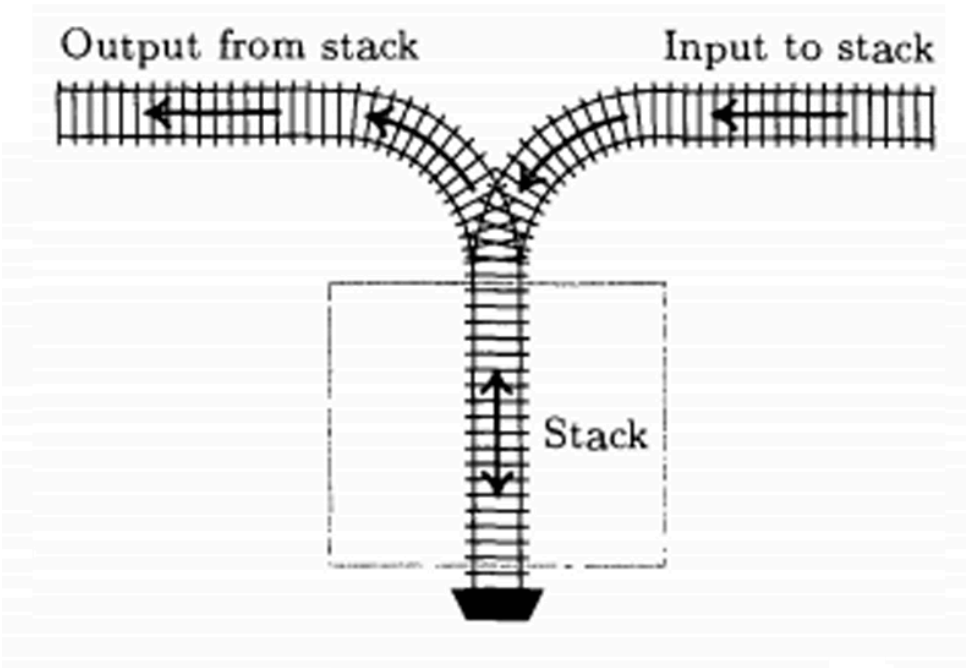


Fig. A stack represented as a railway switching network.

As a result of these operations the original order of the cars, 1234, has been changed into 2431. If there are six railroad cars numbered 123456, Which of the following statements is/are correct?

A. They can't be permuted into the order 154623

B. They can be permuted into the order 254613

C. They can't be permuted into the order 654123

D. They can be permuted into the order 325641

Your Answer: | Correct Answer: A;C;D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #6 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

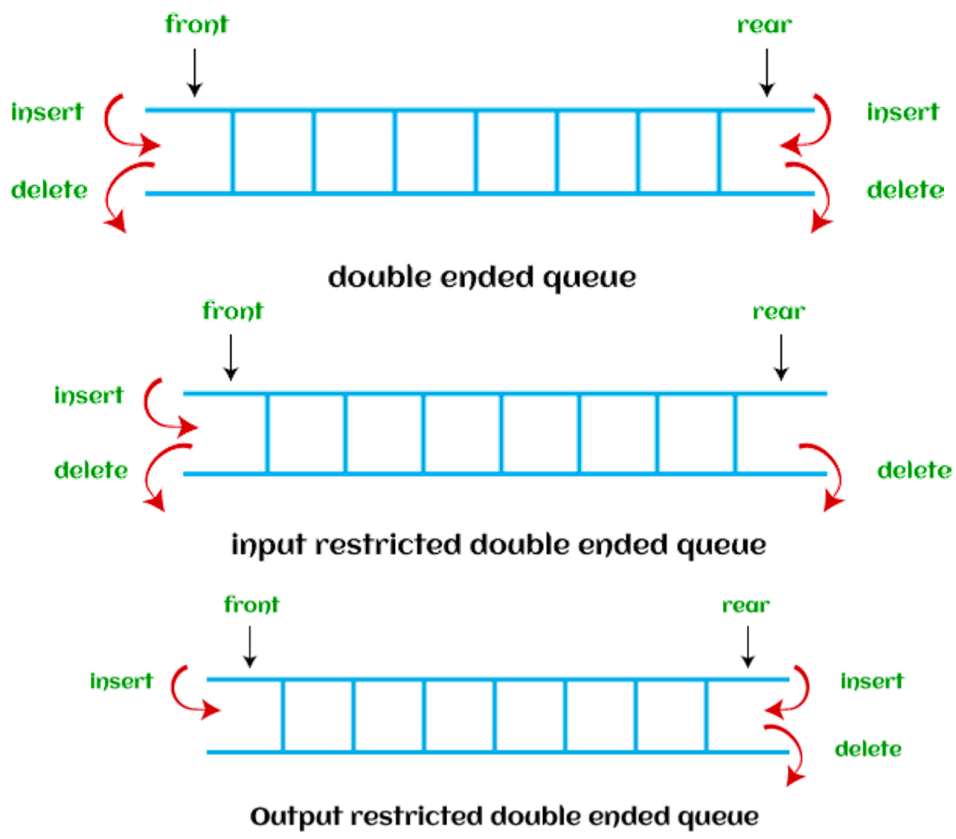
A deque ("double-ended queue") is a linear list for which all the insertions and deletions (and usually all accesses) are made at the ends of the list.

The difference between Queue and Deque is that it does not follow the FIFO (First In, First Out) approach. The second feature of Deque is that we can insert and remove elements from either front or rear ends.

An output-restricted deque is a linear list in which deletions are allowed to take place at only one end but insertion is allowed at both ends.

An input-restricted deque is a linear list in which items may be inserted at one end but removed from either end; clearly, an input-restricted deque can operate either as a stack or as a queue if we consistently remove all items from one of the two ends.

The figure shows types of queues pictorially.



An output-restricted deque can be operated as

A. Only Stack
B. Only Queue
C. Either as a stack or as a queue
D. Neither as a stack nor as a queue

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #7 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Suppose, you have proposed an optimal algorithm which uses extra stack(s) to design a stack such that GetMinimum() should be $O(1)$ for n number of elements.

Consider the following statements:

- $\mathrm{S1}$: Minimum number of extra stack(s) : 1
- $\mathrm{S2}$: Time Complexity : $O(1)$
- $\mathrm{S3}$: Space Complexity : $O(n)$

Which of the following is/are correct?

A. Both $\mathrm{S1}$ and $\mathrm{S2}$ are correct but not $\mathrm{S3}$
B. Both $\mathrm{S1}$ and $\mathrm{S3}$ are correct but not $\mathrm{S2}$
C. Both $\mathrm{S2}$ and $\mathrm{S3}$ are correct but not $\mathrm{S1}$
D. $\mathrm{S1}$, $\mathrm{S2}$ and $\mathrm{S3}$ are correct

Your Answer: | Correct Answer: D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #8 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

We have an implementation that supports the following operations on a stack (in the instructions below, s is the name of the stack).

- $\mathrm{isempty(s)}$: returns True if s is empty, and False otherwise.
- $\mathrm{top(s)}$: returns the top element of the stack, but does not pop the stack; returns null if the stack is empty.

- $\text{push}(\text{s}, \text{x})$: places x on top of the stack.
- $\text{pop}(\text{s})$: pops the stack; does nothing if s is empty.

Consider the following code:

```
push_pop (x) :
    s=empty
    for i=1 to length ( x ) :
        if ( x [i] == '(' ) :
            push ( s , x [ i ] )
        else :
            while ( top (s)== ' ( ' ) :
                pop ( s )
            end while
            push ( s , ' ) ' )
            push ( s , ' ( ' )
        end if
    end for
    while not isempty ( s ) :
        print top(s)
        pop (s)
    end while
```

What is the output of this program when $\text{push pop}(\text{"((((()))))((((")}$ is executed?

A.)())((((
B. ((((()(
C. ((((()))
D.)())(())

Your Answer: | Correct Answer: C | Not Attempted | Time taken: 00min 00sec | Discuss

Q #9 | Multiple Choice Type | Award: 2 | Penalty: 0.67 | Data Structures

Consider the following queue of characters, where QUEUE is a circular array which is allocated 6 memory cells with indexing starting from 1.
Consider the below status of Queue where Front and Rear stores 2 and 4 respectively.

	1	2	3	4	5	6
Current Status of Queue:		A	C	D		
		Front		Rear		

Five sequences of operations have been performed on the current status of queue as described below.

I. 'F' is added to the queue
II. Two items are removed
III. 'K', 'L' and 'M' are added
IV. Two items are deleted
V. 'R' is added to the queue

What will be the final value of Front and Rear respectively?

A. Front $= 2$, Rear $= 5$
B. Front $= 4$, Rear $= 5$
C. Front $= 6$, Rear $= 2$
D. Front $= 6$, Rear $= 3$

Your Answer: | Correct Answer: D | Not Attempted | Time taken: 00min 00sec | Discuss

Q #10 | Multiple Select Type | Award: 2 | Penalty: 0 | Data Structures

A stack of int is implemented using an array as the following data type:

```
#define SIZE 20
typedef struct {
    int data[SIZE];
    int top;
} Stack;
```

Fill up the missing codes in the PUSH, POP, and TOP operations of the Stack.

Ignore underflow or overflow of stack in case of pop and push respectively. You can assume that underflow or overflow are handled before calling pop or push.

```
void Push(Stack *s, int d) {
    // line X
}
void Pop(Stack *s) {
    // line Y
}
int Top(Stack *s) {
    // line Z
}
```

A. `line X should be s->data[++s->top] = d;`

B. `line Y should be --s->top;`

C. `line Z should be return s->data[s->top]`

D. None of these

Your Answer: | Correct Answer: A;B;C | Not Attempted | Time taken: 00min 00sec | Discuss

Copyright & Stuff